

JS

**“ Learn Programming For
Beginners – Free Full Course “**

#3 – Javascript





JS

Programming

Programming is the way we give instructions to a computer to perform specific tasks.

The instructions must be **correct**, **well-structured**, **follow rules** of the programming language being used.



Why Javascript ????

- The Programming Language of the web
- Free
- Easy to Understand
- Versatile





History Javascript

1992

**National Center for Supercomputing Applications (NCSA),
University of Illinois at Urbana–
Champaign developed Mosaic,**

1994

**The Mosaic team created Netscape and
launched Netscape Navigator**

1995

**Brendan Eich get hired
Created A New language inspire by
Java + Scheme + Self = Mocha (in just 10
days!)
Mocha -> Livescript -> Javascript**

1996

**Netscape sought Standardization
W3C rejected the proposal, so Netscape
turned to ECMA International**



ECMAScript

ECMAScript 1 (1997) -> First Edition

ECMAScript 2 (1998) -> Only Editorial Changes

ECMAScript 3 (1999) -> Regex, Try/Catch, Switch, do-while

ECMAScript 4 -> Never Released

ECMAScript 5 (2009) -> use strict, JSON Support, String.trim(), Array.isArray(), Array iteration methods, Allows trailing commas for object literals

ECMAScript 6 (2015) -> Added let and const, Default parameter values, Array.find(), Array.findIndex()



How To Add

```
<script src="./script.js"></script>
```

```
<script>//your code </script>
```

it can be placed in the `<body>`, or in the `<head>` section of an HTML page



Output

```
console.log("output")
```

```
document.getElementById("demo").  
innerText = "demo"
```

```
document.write("output")
```

```
window.alert("output")
```



Variable

Variable = containers for storing data

Const

Let

Var (obsolete)

Automatically
(obsolete)

Rules Variable :

- Variable can contain letter, digits, underscores, and dollarsign
- Variable must begin with a letter, a \$ sign or an underscore
- Hyphens are not allowed in Javascript
- JavaScript programmers tend to use lower camel case

example :

```
const yourName = "Josse Surya Pinem"
```




8 DataTypes

String = A Text of Characters enclosed

example :

```
const person = "Josse Surya Pinem";
```

Number = A number representing a mathematical value

example :

```
const length = 18.95;
```

Bigint = A number representing a large integer

example :

```
const x = 1234567890123456789012345n;
```

Boolean = A data type representing true of false

example :

```
const correct = true;
```



Object = A collection of key-value pairs of data

example :

```
const employee = {  
  name: "Josse Surya Pinem",  
  position: "Software Developer"  
};
```

Undefined = A primitive variable with no assigned value

example :

```
let x;
```

Null = A primitive value representing object absence

example :

```
let x = null
```

Symbol = A unique and primitive identifier

example :

```
const x = Symbol()
```



Operator

+ = Addition

- = Subtraction

*** = Multiplication**

/ = Division

% = Modulus(remainder)

-- = decrement

++ = increment



== = equal to

=== = equal value dan equal data type

!= = not equal

!== = not equal value or not equal data type

&& = and

|| = OR



$>$ = greater than

$>=$ = greater than or equal to

$<$ = less than

$<=$ = less equal than or equal to



Conditionals

Conditional Statements allow us to perform different actions for different conditions

Conditional Statements run different code depending on true or false conditions

Syntax

```
If (condition1) {  
    // code to execute if condition1 is true  
} else if (condition2){  
    // code to execute if the condition1 is false  
    and condition2 is true  
} else {  
    // code to execute if the condition1 is false,  
    and condition2 is false  
}
```



```
switch(expression){  
  case x:  
    //code block  
    break;  
  case y:  
    //code block  
    break;  
  default:  
    //code block  
}
```

condition ? expression1 : expression2



Loops

Loops are handy, if you want **to run the same code over and over again**, each time with a different value

script.js

```
console.log(1);  
console.log(2);  
console.log(3);  
console.log(4);  
console.log(5);
```



script.js

```
for (let i = 1; i < 6; i++){  
  console.log(i);  
}
```




script.js

```
let i = 0;  
do{  
  console.log(i)  
  i++;  
}  
while ( i < 6 );
```

script.js

```
let i = 0;  
while (i < 6){  
  console.log(i)  
  i++;  
}
```



Function

Functions are fundamental building blocks in all programming
Functions are reusable block of code designed to perform a particular task
Functions are executed when they are called invoked

Syntax :

```
function myFunction(param){  
  // code to be executed  
}
```

Syntax :

```
const myFunction = (param) =>{  
  // code to be executed  
}
```

```
// call or invoke function  
myFunction(args)
```



Array

An Array is an object type designed for storing data collections

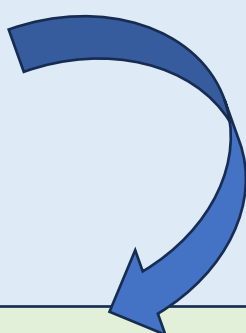
example:

```
const lecturer1 = "josse"
```

```
const lecturer2 = "surya"
```

```
const lecturer3 = "pinem"
```

```
const lecturer4 = "john"
```



```
const lectures = ["josse",  
"surya", "pinem", "john"];
```



Array Method

`push()` = method adds new items to the end of an array

`filter()` = creates a new array with every element in an array that pass a test

`map()` = creates a new array with the result of calling a function for each array element



`splice()` = add or removes array elements

`array.splice(index, count, item1, ..., itemX)`

Index = position array

count = many of items to be placed or removed

Item1 = element to be added

`reverse()` = reverses the order of the elements in an array

`sort()` = sort the elements of an array



Objects

An Object is a variable that can hold many variables

Objects are collections of **key-value pairs**, where each key (known as property name) has a value

Objects can describe anything like houses, cars, people, animals, or any other subjects
Objects are container for **Properties and Method**

example

```
const person = {
```

```
  firstName : "Josse",  
  lastName  : "Pinem",  
  id: 1,
```

Property

```
  fullName: function(){  
    return this.firstName + "  
    + this.lastName
```

Method

```
}
```

```
}
```



DOM

(Document Object Model)

The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

The HTML DOM is a standard for how to find and manipulate HTML elements
– Josse

Finding HTML elements :

`document.querySelector()`

`document.getElementById()`

`document.getElementsByTagName()`

`document.getElementsByClassName()`

Changing elements :

`element.innerHTML = new html content`

`element.style.property = new style`

`element.classList.add|remove|toggle(“selectorclass”)`

Adding Events Handlers :

`element.addEventListener(“click”,
myFunction)`



Finding HTML elements :

`document.querySelector()`

`document.getElementById()`

`document.getElementsByTagName()`

`document.getElementsByClassName()`

Changing elements :

`element.innerHTML = new html content`

`element.style.property = new style`

`element.classList.add|remove|toggle(“selecto
rclass”)`

Adding Events Handlers :

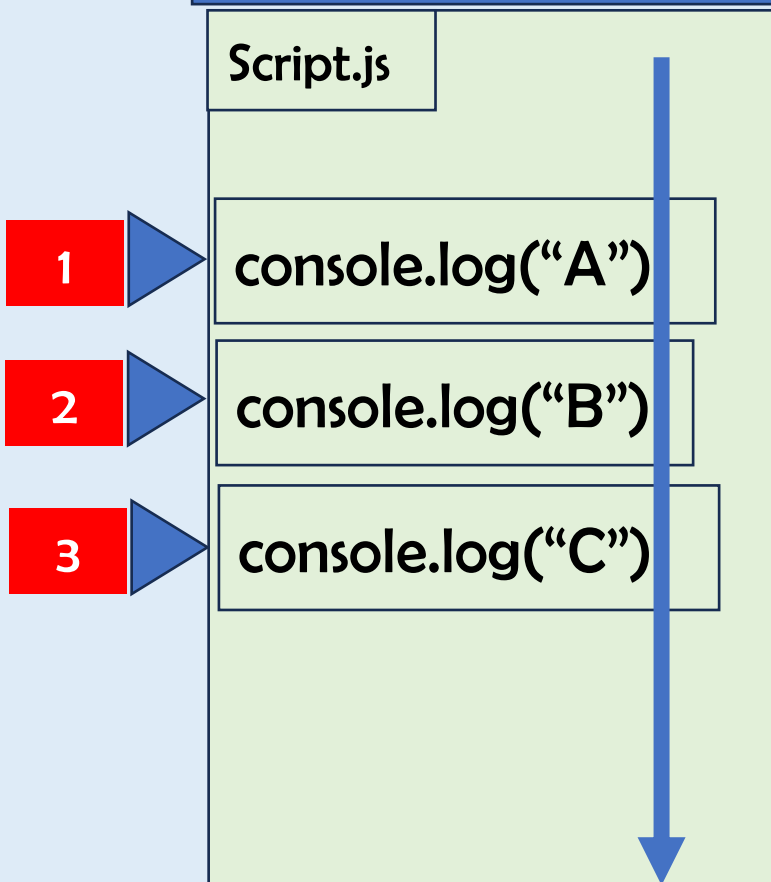
`element.addEventListener(“click”,
myFunction)`

Adding Event Handlers Drag :

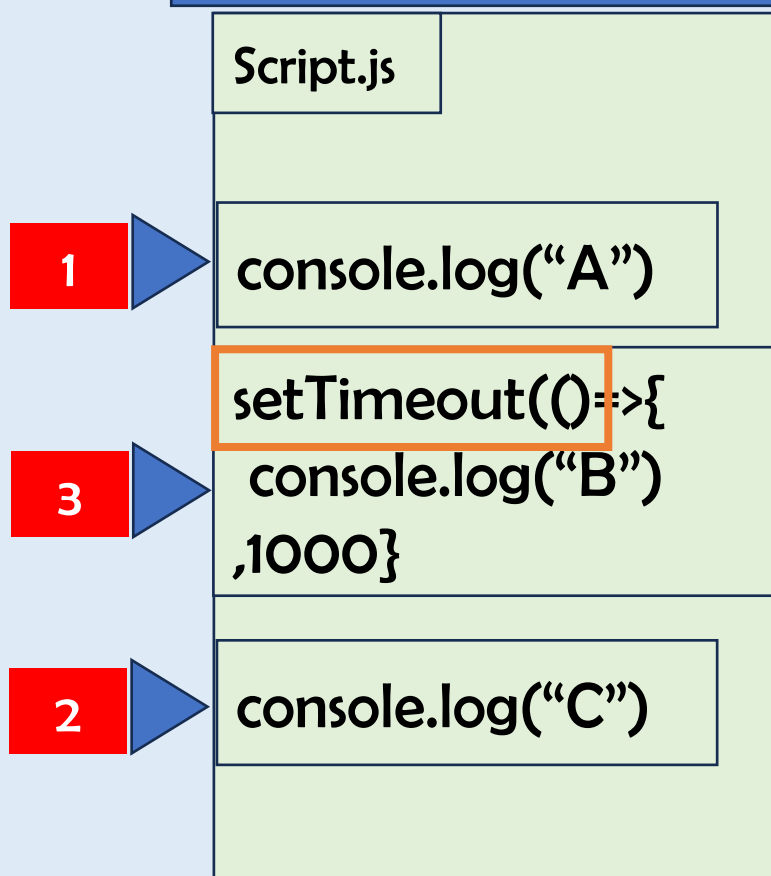
1. dragstart
2. dragover
3. dragenter
4. dragleave
5. drop
6. dragend



Synchronous



Asynchronous





Callback

A callback is a **function passed as an argument** to another function

```
const getData = (callback) => {  
  setTimeout(()=>{  
    callback("Data Loaded")  
  }, 1000);  
}  
getData((result))=>{  
  console.log(result)  
});
```

example



```
getUser(id, (user) => {  
  getPosts(user, (posts) => {  
    getComments(posts, (comments)  
    => {  
      processComments(comments, (result)  
      =>{console.log(result)}};  
    });  
  });  
});
```

Callback Hell !!!



```
getUser(id)  
  .then(user => getPosts(user))  
  .then(posts => getComments(posts))  
  .then(comments =>{  
    processComments(comments)})  
  .then(result => console.log(result))  
  .catch(error => console.error(error));
```





Promise

A object represents a value that will be available now, later, or never

States = pending, fulfilled, rejected

then

catch



Example Promise & Fetch API

```
const getUser = () => {  
  fetch(`https://fakestoreapi.com/users`)  
    .then(res => response.json())  
    .then(res => console.log(res))  
    .catch(err => console.log(err))  
}  
getUser()
```

Promise Fetch API with async await

```
const getUser = async () => {  
  try {  
    const res = await  
      fetch(`https://fakestoreapi.com/users`)  
    const data = await response.json()  
    console.log(data)  
  } catch (err) {  
    console.error(err)  
  }  
}  
getUser()
```



References :

<https://www.w3schools.com/js/default.asp>



THANK YOU !

Don't forget to visit

<https://josserich.github.io>

